Explaining the New URPX Condition Logic Model

Instead of hiding logic in vague value fields or string expressions, the new model provides structured, machine-readable condition modeling

Una Smithsimon

What This Replaces

A look back at the limitations of the v0.1.2 condition logic model.

The Legacy v0.1.2 Model

- Our v0.1.2 model represented complex conditions with a series of string values including `conditionType` and `conditionValue`.
- Combinations of conditions with boolean logic were expressed with another string property, `logicExpression`.

Why the old pattern was risky

The main issue was loss of machine-readable meaning and execution clarity.

Not machine-readable

Not executable with confidence

Risk of misinterpretation

What the new model provides



Structured Condition Modeling



Nested Rule Calculation



Condition Combination Structure

The new Eligibility Rule model with Conditions

EligibilityRule

Business rule container

Holds context such as customer or service-point profile and conditions.

Condition

One atomic business test

Examples:
• averageMonthlyUsage > 500
• load factor > 0.4

ConditionLogic

How multiple conditions combine

Boolean only:
AND, OR, NOT

Even a single-condition rule still uses this layer.

Outcome

A consistent applicability pattern that is easier to compute, validate, and explain.

Condition Model

Inside one Condition

A Condition represents one independently evaluable predicate.

That means it can include:
- a comparison operator
- ordered operands
- nested arithmetic or expression prep through OperationNode when needed

Example: load factor > 0.4 modeled with a nested calculation as (kWh usage / kW usage) > 0.4

Condition Example

Old URPX v0.1.2 model

```
{
  "@type": "urpx:Condition",
  "urpx:operator": { "@id": "urpx:greaterThanOperator" },
  "urpx:conditionType”: “load factor”,
  “urpx:conditionValue”: “0.4”
}
```

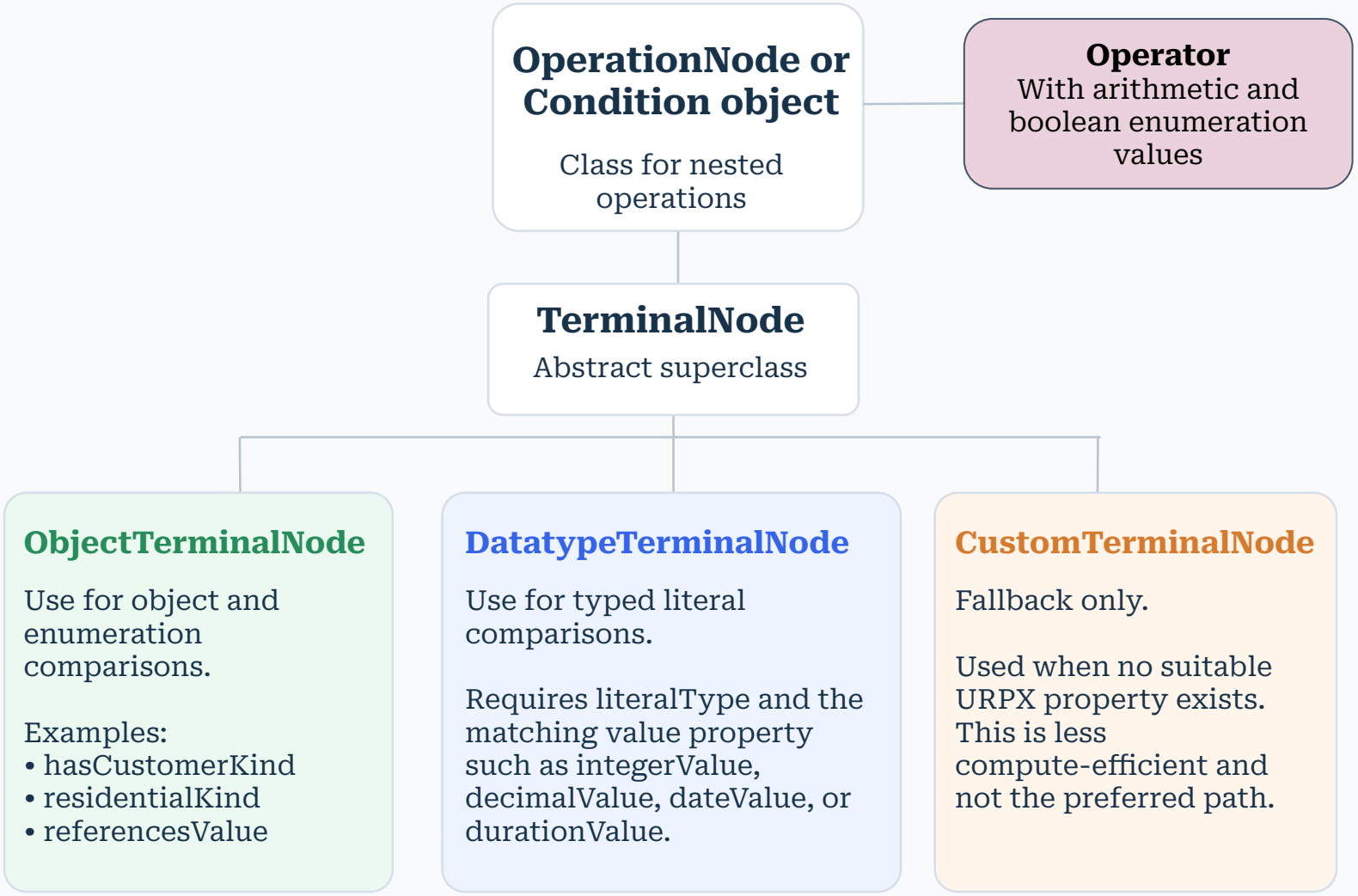
New URPX v0.2.0 model

```
{
  "@type": "urpx:Condition",
  "urpx:operator": { "@id": "urpx:greaterThanOperator" },
  "urpx:leftOperand": {

    "@type": "urpx:OperationNode",
    "urpx:operator": { "@id": "urpx:divideOperator" },
    "urpx:leftOperand": {
      "@type": "urpx:ObjectTerminalNode",
      "urpx:referencesMetricInput": { "@id": "urpx:mi-energy" }
    },
    "urpx:rightOperand": {
      "@type": "urpx:ObjectTerminalNode",
      "urpx:referencesMetricInput": { "@id": "urpx:mi-demand" }
    }
  },
  "urpx:rightOperand": {
    "@type": "urpx:DatatypeTerminalNode",
    "urpx:literalType": { "@id": "urpx:decimalDatatype" },
    "urpx:decimalValue": 0.4
  }
}
```

How values become explicit

URPX v0.2.0 replaces string condition values with typed terminal-node patterns.



Condition Logic

Inside ConditionLogic

ConditionLogic combines already-defined conditions using boolean operators only.

ConditionLogic determines the applicability of Condition definitions and must be used when any Condition objects are present, even for a single Condition.

Example: (Condition A AND Condition B) OR Condition C

The logic layer is not a general computation graph, it is a boolean combination graph.

Condition Logic Example

URPX v0.1.2 Condition model

“urpx:logicExpression”: “condition-a AND condition-b”

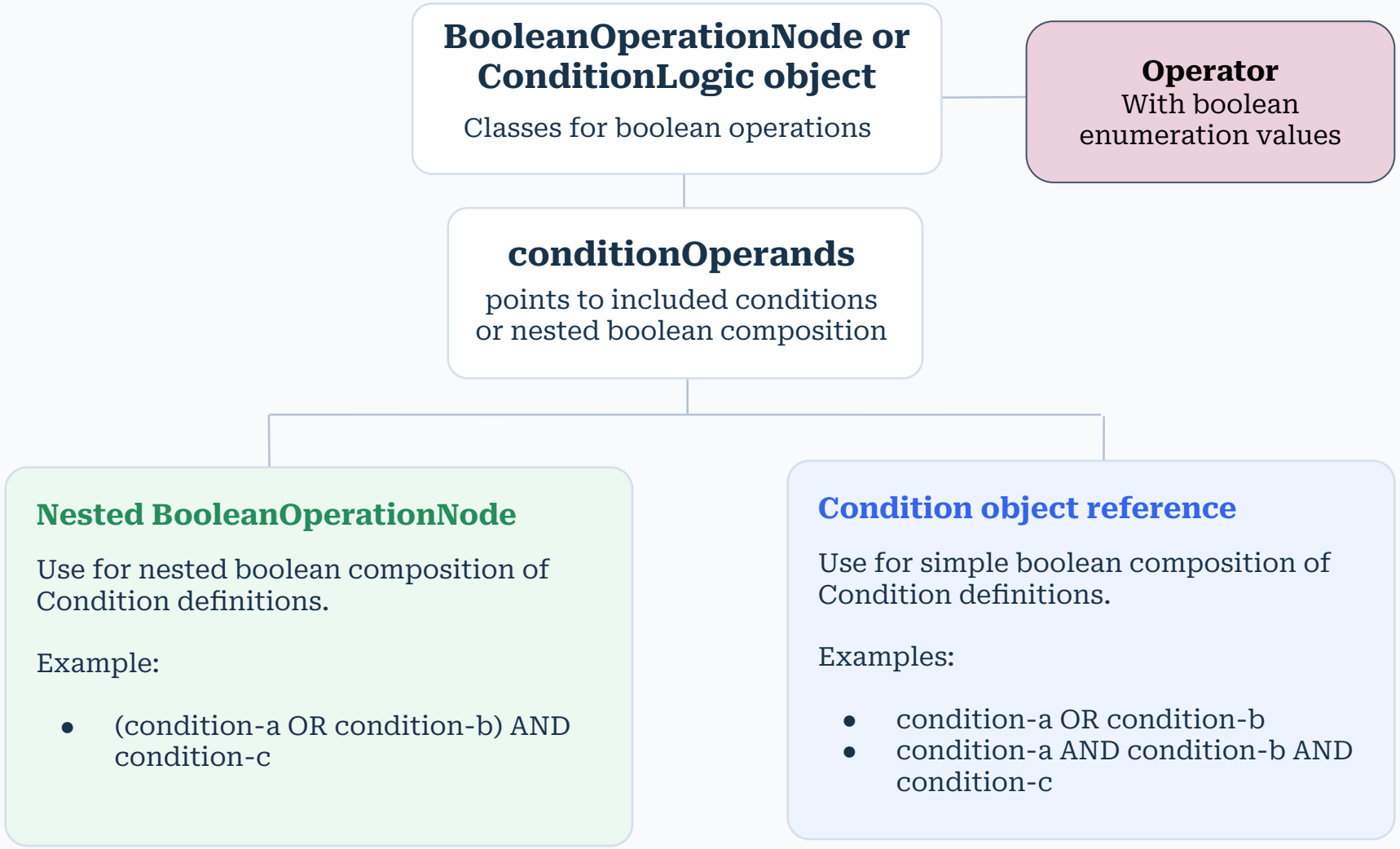
URPX v0.2.0 Combined Applicability of Multiple Conditions

```
{
  "@type": "urpx:ConditionLogic",
  "urpx:operator": { "@id": "urpx:orOperator" },
  "urpx:conditionOperands": [
    {
      "@type": "urpx:BooleanOperationNode",
      "urpx:operator": { "@id": "urpx:andOperator" },
      "urpx:conditionOperands": [
        { "@id": "urpx:condition-a" },
        { "@id": "urpx:condition-b" }
      ]
    },
    { "@id": "urpx:condition-c" }
  ]
}
```

URPX v0.2.0 Applicability of a Single Condition

```
{
  "@type": "urpx:ConditionLogic",
  "urpx:operator": { "@id": "urpx:andOperator" },
  "urpx:conditionOperands": [
    { "@id": "urpx:condition-customer-kind" }
  ]
}
```

How Condition applicability becomes explicit



Real-world example: PGE E-ELEC eligibility

Tariff eligibility language

The schedule applies to residential customers with one or more qualifying electric technologies, including:

- electric vehicle charging
- energy storage
- electric heat pump

The tariff text also notes a special case for Net Billing Tariff customers.

What the current JSON-LD does

The v0.1.2 test data file models eligibility with attached Condition objects plus a string logicExpression:

“storage OR EV charging OR heat pump OR NBT”

That is readable, but it keeps important execution meaning in a string.

How v0.2.0 would structure these conditions

The same business rule could be expressed with:

- atomic Condition definitions
- a ConditionLogic object with explicit boolean structure

That preserves the business meaning while making the execution structure unambiguous.

Resources for review

Review the old v0.1.2 Condition representation model:

- in the urpx repository, on the main branch:
  - ontology/urpx-ontology.ttl defines the old Condition model, including use of logicExpression for boolean combinations of conditions.
  - shacl/SHACL.ttl defines constraints for old Condition model
  - test-cases/pge-e-elec-2026-01-01 contains a test case with a tariff book PDF and a JSON-LD URPX rate plan showing the old Condition model

Review the new v0.2.0 proposed Condition representation model:

- in the urpx repository, on branch v0.2.0-candidate:
  - ontology/urpx-ontology.ttl defines new and updated Classes and properties used for condition modeling
  - shacl/SHACL.ttl defines constraints for new Condition and ConditionLogic model
  - test-cases/pge-e-elec-2026-01-01 contains a test case with a tariff book PDF and a JSON-LD URPX rate plan showing new condition model implementation